

Cursor

Cursor is an AI-assisted code editor

The AI agent can:

- Read and write files on your machine
- Edit many files at once
- Run shell commands

You work in a normal editor

The agent works in the same project folder

Massive productivity boost for editing source code

Cursor is used by tens of thousands of companies, including NVIDIA, Adobe, and Stripe

In June SpaceX announced an agreement to acquire the company behind Cursor for \$60 billion

Considerations

Out of the box, Cursor massively speeds up writing code and docs

A single prompt can generate thousands of lines of content across many files

That speed creates two problems we must control:

1. **Access risk** - The agent can reach local drives, files, and data
That raises the chance of data loss, breach, or incident
2. **Quality risk** - Unguided content production
Drift between sessions, projects, and users
Weak trust in what was produced

Doing so without proper controls is immature tool use, not a flaw in Cursor itself

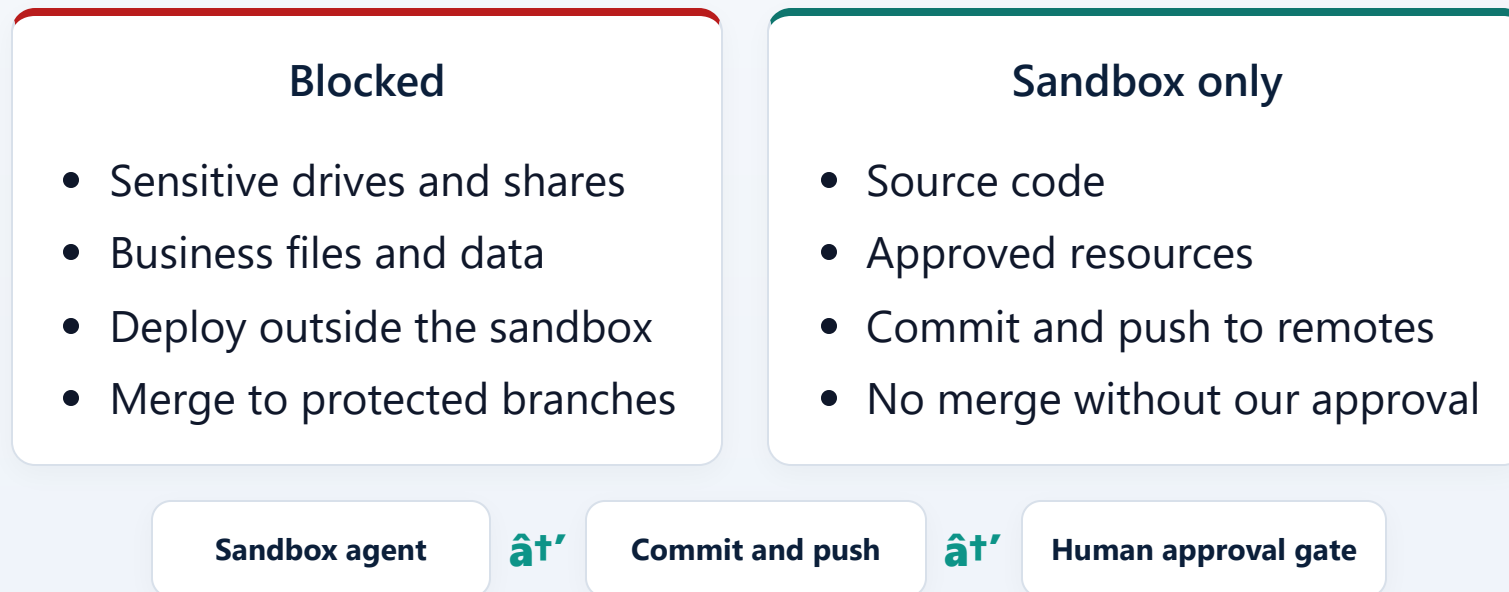
Environment isolation

Cursor can edit local files and run shell commands

It is not acceptable for business users with access to sensitive drives, files, or data to run a local agent on their normal machine

A local agent with shell access can do as much damage as a person, but much faster

Mitigation - Isolate the agent from everything we do not want it to see or touch



All changes to source of truth stay behind our approval process

The agent cannot deploy code anywhere outside the sandbox

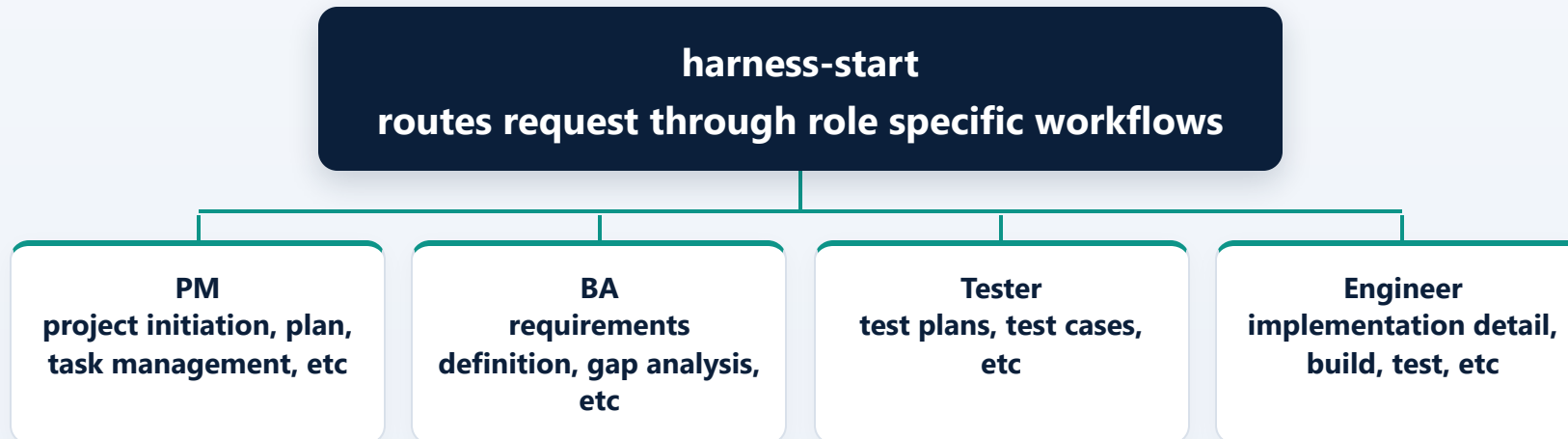
Agent Harness

Design led by official guidance and trusted open source projects

We introduce an agent harness to control the available context and the workflows which the agents perform

- Always-on rules enforce behaviour on every prompt
- Skills define repeatable entry points that direct the agent to a set of workflows
- Workflows define the exact procedure for specific tasks

Example: user starts a new project with our main skill `/harness-start`



Knowledge bases

In order to supply the agent with the best possible context - we maintain highly structured, highly searchable knowledge bases with deep domain expertise

These store a hierarchy of information from high level principles to low level implementation detail

We also enforce role specific workflows, templates, and guidance to ensure consistency in agent process

Long term goal - every business unit maintains a complete knowledge base which is approved and made shareable via source control

Knowledge base	Domain
Change	PM, BA, Tester roles and project documents
Engineering	Design principles, implementation details, and coding standards
Systems	Platforms, tools, integration, and business processes

Change knowledge base

Purpose - Controlled approach to project and change management

Content - Four topic areas, over 400 notes: business analysis, project management, test analysis, and the project document pack

Source - BCS open project methods material

Example work - On project start the agent fills a defined pack:

Document	Owner
Initiation charter	PM
Problem frame	BA
Requirements catalogue	BA
Delivery plan and RAID	PM
Test conditions and cases	Tester
UAT sign-off	Tester

Engineering knowledge base

Purpose - Controlled approach to engineering supporting high level design through low level code patterns

Content - 12 topics, over 400 notes

Includes languages, data and SQL, delivery, containers, infrastructure, APIs, security, and architecture

Sources - OWASP, DORA, Kimball, Fowler, OpenAPI, Docker, Kubernetes, and prior project repos

Our notes win when house choices differ

Example work - Warehouse ODS historisation

Principle - Landing is reloadable

History is durable

Serving marts must not become a second history engine

Standard - ODS owns Type-2 history

Dimensions carry effective dates, current flag, delete flag, and row hash

Pattern - SQL merge closes the current row when the hash changes, then inserts the new version

```
UPDATE ods.Customer
SET   effective_to = @as_of, is_current = 0
WHERE business_key = @key AND is_current = 1
      AND row_hash <> @new_hash;

INSERT INTO ods.Customer (business_key, effective_from, is_current, row_hash)
VALUES (@key, @as_of, 1, @new_hash);
```

Systems knowledge base

Purpose - Highly detailed documentation of business systems and processes

Content - 10 platform families, over 170 notes

Includes cloud, databases, source control hosts, CRM, and service desk

Example work - Zoho CRM REST API v7

- Endpoint: `https://www.zohoapis.{dc}/crm/v7/...` - data centre sets the host
- Auth: OAuth 2.0 access token on every call
- Required header: `Authorization: Zoho-oauthtoken <access_token>`
- Write calls also send `Content-Type: application/json`
- Access tokens last one hour by default
Refresh tokens until revoked

Structure and indexing

Knowledge is layered
High level choices drive lower level detail

```
knowledge-base/  
  maps/           indexes  
  principles/     why  
  standards/      the rules  
  guidance/       how to apply  
  workflows/      agent steps  
  examples/       worked shapes
```

Example index format - Atlases list topics and point to notes:

Family or theme	Atlas	Points to
Warehouse	warehouse atlas	SQL layers, SCD2 dims, facts
Security	security atlas	Auth and secure coding notes
Business analysis	BA atlas	Framing and requirements notes

Agent path: open the index, then only the notes needed

We run coded tests for index health

Git/DevOps control

Agent harness and knowledge bases have changes tracked by Git
Shared remote source is stored in DevOps, with changes gated by approval - the same standard we apply to all source code
That gives version control, audit trail, and rollback
Knowledge bases must pass CI health checks on structure and indexing before merge

Repo	Owns
agent-harness	Shared control hub
change-knowledge-base	PM, BA, Tester methods
engineering-knowledge-base	Build and test craft
systems-knowledge-base	Platforms and processes
Product repos	Source code and docs

Rollout

Suggest data team first

Estimate 2-4 weeks to get fully integrated and iron out any kinks

Build out the structure specific to our environment and define the proper source control process

Then bring in IT and Change with remit on their specific knowledge bases

Zak and Sam as Cursor SMEs

We own the agent harness and knowledge base structure

We can block changes that do not meet our standards